Dear Students,

Below are the **70 Operating System Mini Projects (30 Marks)** with a **short project description**, mapped to the syllabus topics.
Objective of these projects are to reinforce the operating system concepts covered in your syllabus while solving practical, real-world problems.

Module 1: Introduction to Operating Systems

No.	Project Title	Short Description	OS Concepts
1	CPU and Memory Usage Monitor	Develop a desktop application that displays real-time CPU, RAM, and disk utilization of the computer.	OS Services, Resource Management
2	Operating System Information Dashboard	Create a tool that displays OS version, architecture, processor, memory, and storage information.	OS Services, System Calls
3	Task Manager Clone	Build a simplified Task Manager to display and terminate running processes.	Process Management
4	Laptop Resource Utilization Analyzer	Develop software that analyzes CPU, RAM, and battery usage while applications are running.	Resource Allocation
5	Smart Home Device Resource Monitor	Monitor CPU and memory usage of IoT devices connected to a smart home network.	OS Resource Management
6	Battery Health Monitoring Application	Develop an application that monitors battery health, charging cycles, and remaining backup time.	OS Services
7	Application Startup Time Analyzer	Measure and compare the startup time of commonly used applications.	Performance Analysis
8	System Boot Performance Analyzer	Analyze boot time and identify services that delay system startup.	OS Initialization
9	Desktop Performance Monitoring Tool	Create a dashboard showing CPU, RAM, network, and disk usage graphs.	Resource Management

No.	Project Title	Short Description	OS Concepts
10	Operating System Comparison Tool	Compare Linux and Windows based on CPU scheduling, memory, and file systems.	OS Architecture

Module 2: OS Principles

No.	Project Title	Short Description	OS Concepts
11	Process Creation Visualizer	Visualize process creation and state transitions while executing programs.	PCB, Process States
12	Thread Management Simulator	Demonstrate user-level and kernel-level thread execution using animations.	Threads
13	Multi-threaded File Downloader	Develop a downloader that uses multiple threads to download file segments simultaneously.	Multithreading
14	Simple Command Line Shell	Implement a mini Linux shell supporting basic commands such as ls, pwd, mkdir, and cp.	System Calls
15	System Call Monitoring Tool	Display system calls generated while opening files or executing applications.	System Calls
16	Student Attendance Process Tracker	Represent each student login as a process and monitor attendance sessions.	Processes
17	Hospital Patient Queue Simulator	Model patient registration and treatment as processes managed by scheduling algorithms.	Scheduling
18	Multi-user Login Simulation	Develop a login system supporting multiple users with access privileges.	Protection
19	Simple Linux Command Executor	Create software to execute Linux commands through a graphical interface.	System Calls
20	Online Examination Process Manager	Manage student sessions as independent processes during online examinations.	Process Management

Module 3: Scheduling

No.	Project Title	Short Description	OS Concepts
21	CPU Scheduling Simulator	Implement FCFS, SJF, Priority, and Round Robin scheduling with Gantt charts.	CPU Scheduling
22	Hospital OP Queue Scheduler	Prioritize emergency patients using Priority Scheduling.	Priority Scheduling
23	Restaurant Order Scheduling	Schedule food preparation orders using FCFS and SJF algorithms.	Scheduling
24	Airport Runway Scheduler	Allocate runway access based on aircraft priority.	Priority Scheduling
25	Printer Queue Management	Manage print jobs using FCFS scheduling.	Scheduling
26	Traffic Signal Scheduler	Control traffic lights using Round Robin scheduling principles.	Real-Time Scheduling
27	Computer Lab Scheduler	Allocate computer systems to students using scheduling algorithms.	Scheduling
28	Deadlock Detection Simulator	Simulate deadlock situations and identify blocked processes.	Deadlock Detection
29	Banker's Algorithm Visualizer	Develop an interactive Banker's Algorithm simulator for resource allocation.	Deadlock Avoidance
30	Cloud Resource Allocation Simulator	Allocate processors and memory dynamically among cloud users.	Resource Allocation

Module 4: Concurrency

No.	Project Title	Short Description	OS Concepts
31	Railway Ticket Booking System	Prevent multiple users from booking the same seat simultaneously.	Synchronization
32	ATM Cash Withdrawal Simulator	Prevent race conditions when multiple users access the same bank account.	Critical Section
33	Online Banking Transaction System	Synchronize deposits and withdrawals using mutex locks.	Mutex, Semaphore
34	Library Book Reservation System	Allow only one student to reserve a book at a time.	Synchronization
35	Shared Printer Management System	Manage printer access using semaphores.	Semaphore
36	Producer-Consumer Animation	Visualize producer-consumer synchronization using shared buffers.	Producer-Consumer

No.	Project Title	Short Description	OS Concepts
37	Dining Philosophers Simulation	Implement Dining Philosophers using semaphores to avoid deadlock.	Synchronization
38	Traffic Junction Synchronization	Synchronize vehicle movement at intersections using mutexes.	Synchronization
39	Collaborative Document Editor	Prevent simultaneous editing conflicts among multiple users.	Locks
40	Chat Application using IPC	Exchange messages between two processes using pipes or shared memory.	IPC

Module 5: Memory Management

No.	Project Title	Short Description	OS Concepts
41	Memory Allocation Simulator	Implement First Fit, Best Fit, and Worst Fit memory allocation.	Memory Allocation
42	Virtual Memory Simulator	Translate logical addresses to physical addresses using paging.	Virtual Memory
43	Page Replacement Visualizer	Compare FIFO, LRU, and Optimal page replacement algorithms.	Page Replacement
44	Memory Fragmentation Analyzer	Calculate internal and external fragmentation.	Memory Management
45	TLB Address Translation Simulator	Simulate virtual-to-physical address translation using TLB.	TLB
46	RAM Usage Monitor	Monitor memory consumption of running applications.	Memory Management
47	Cache Performance Analyzer	Compare cache hit and miss rates for different workloads.	Memory Hierarchy
48	Working Set Calculator	Determine the working set size of active processes.	Working Set
49	Demand Paging Simulator	Simulate page loading only when pages are referenced.	Demand Paging
50	Memory Leak Detection Tool	Identify programs consuming excessive memory over time.	Memory Management

Module 6: Virtualization & File Systems

No.	Project Title	Short Description	OS Concepts
51	Virtual Machine Resource Monitor	Monitor CPU and memory usage of virtual machines.	Virtualization
52	Container Monitoring Dashboard	Monitor Docker containers and their resource usage.	Containerization
53	Cloud VM Allocation System	Allocate virtual machines based on user requirements.	Virtualization

No.	Project Title	Short Description	OS Concepts
54	Simple File Manager	Develop a graphical application to create, move, rename, and delete files.	File System
55	Duplicate File Finder	Identify duplicate files based on file size and content.	File Management
56	Disk Space Analyzer	Visualize disk usage of folders and files.	Storage Management
57	Journaling File System Demonstrator	Demonstrate file recovery using journaling concepts.	Journaling
58	File Compression Utility	Compress and extract files using standard compression techniques.	File Management
59	File Backup and Restore System	Automatically back up selected folders and restore them when required.	File System
60	Recycle Bin Recovery Tool	Recover accidentally deleted files before permanent deletion.	File Management

Module 7: Storage Management, Protection & Security

No.	Project Title	Short Description	OS Concepts
61	Disk Scheduling Simulator	Compare FCFS, SSTF, SCAN, and C-SCAN disk scheduling algorithms.	Disk Scheduling
62	USB Access Monitoring Tool	Detect USB insertion/removal and maintain an activity log.	OS Security
63	Password-Based File Locker	Protect confidential files using password authentication.	Authentication
64	Role-Based Access Control System	Provide different access permissions for Admin, Faculty, and Student users.	Access Matrix
65	Secure File Sharing Application	Allow authenticated users to securely share files over a network.	Protection
66	Student Record Access Control System	Restrict student record access using capability-based permissions.	Capability-Based Protection
67	Disk Performance Analyzer	Measure disk read/write speed and compare storage devices.	Disk Management
68	Login Authentication System with Logging	Implement secure login with password verification and activity logs.	System Protection

No.	Project Title	Short Description	OS Concepts
69	File Permission Manager	Develop software to modify read, write, and execute permissions for files.	Access Protection
70	Ransomware Detection Simulation	Simulate suspicious file-encryption behaviour and generate security alerts.	OS Security

Below is a **Recommended Software Tools** table suitable for the **Operating Systems (30 Marks Mini Projects)** listed above.

Category	Recommended Software/Tools	Purpose
Operating Systems	Ubuntu Linux (22.04 LTS or later), Windows 10/11	Development, testing, and system programming
Programming Languages	C, C++, Python, Java	Implementation of OS concepts and mini projects
Integrated Development Environments (IDEs)	Visual Studio Code, Code::Blocks, Eclipse, IntelliJ IDEA Community, PyCharm Community	Coding, debugging, and project development
Compilers	GCC/G++, Clang, MinGW (Windows), OpenJDK	Compiling C, C++, and Java programs
Python Libraries	psutil, threading, multiprocessing, os, shutil, pathlib, tkinter, watchdog	System monitoring, process management, file systems, GUI development
Java Libraries	Java Threads, Java NIO, Java Swing, JavaFX	Multithreading, file management, graphical interfaces
C/C++ Libraries	POSIX Threads (pthread), stdio, stdlib, unistd, sys/stat, semaphore.h	Process management, synchronization, file operations
Database	SQLite, MySQL Community Edition	Store user information, logs, scheduling data, authentication records
Virtualization Software	Oracle VirtualBox, VMware Workstation Player	Virtual machine creation and virtualization experiments
Containerization	Docker Desktop	Container-based virtualization projects
Version Control	Git, GitHub	Source code management and collaboration
File System Utilities	Linux Terminal, File Explorer, Disk Management Tools	File operations and storage management

Category	Recommended Software/Tools	Purpose
Performance Monitoring Tools	htop, top, Task Manager, Resource Monitor, Process Explorer	Monitor CPU, memory, and process performance
Disk Analysis Tools	GParted, CrystalDiskInfo, WinDirStat	Disk partitioning, health monitoring, storage analysis
Network Utilities	Wireshark, netstat, ping, ipconfig/ifconfig	IPC, networking, and communication-related projects
Documentation Tools	Microsoft Word, Google Docs, LaTeX (Optional)	Project report preparation
Diagram Tools	draw.io (diagrams.net), Lucidchart, Microsoft Visio	Flowcharts, UML diagrams, architecture diagrams
Testing Tools	JUnit (Java), unittest (Python), Google Test (C++)	Unit testing and validation
Build Tools	Make, CMake, Maven (Java), Gradle	Build automation and project management

Minimum Hardware Requirements

Component	Recommended Specification
Processor	Intel Core i3 (10th Gen) / AMD Ryzen 3 or higher
RAM	8 GB (16 GB recommended for virtualization projects)
Storage	256 GB SSD (minimum), 512 GB SSD recommended
Operating System	Windows 10/11 (64-bit) or Ubuntu Linux 22.04 LTS
Internet	Broadband connection for downloading libraries and documentation
Display	1366 × 768 resolution or higher

Software Recommendations by Project Category

Project Category	Recommended Software
Process Management	C/C++, GCC, Visual Studio Code, Ubuntu Linux
CPU Scheduling	C/C++, Python, Java, Code::Blocks
Concurrency & Synchronization	C (POSIX Threads), Java Threads, Python threading/multiprocessing
Memory Management	C/C++, Python, Visual Studio Code

Project Category	Recommended Software
Virtualization	VirtualBox, VMware Workstation Player, Docker Desktop
File System Projects	Python, Java, C++, SQLite
Disk Scheduling	C/C++, Python, Visual Studio Code
Security & Protection	Python, Java, SQLite, OpenSSL (optional)
Resource Monitoring	Python (psutil), Task Manager APIs, htop
GUI-Based Applications	Python (Tkinter), Java Swing/JavaFX, C# Windows Forms (optional)

These tools are **free or community editions**, making them appropriate for laboratory environments while covering all the concepts in your Operating Systems syllabus.

Expected Deliverables for Each 30-Mark:

Students should submit a report:

1. **Problem Statement** (1 page)
2. **Objectives** (3–5 objectives)
3. **System Design** (Flowchart/UML/Architecture)
4. **Algorithm or Pseudocode**
5. **Implementation** (C/C++/Java/Python)
6. **Screenshots and Results**
7. **Testing and Validation**
8. **Conclusion and Future Enhancements**
9. **References**

Final Demo to be presented before last working day.